

Document Technique

Cahier des charges

Danjon Simulator – Guillaume Boudaille

Logiciel de simulation visant à chiffrer les possibilités d'un roleplay

1 Présentation du projet

1.1 Vision

Le projet est une application lourde conçue pour simuler, tester et analyser les règles d'un serveur de roleplay. Face à la complexité des systèmes interconnectés, l'outil permet d'abandonner les tests manuels fastidieux au profit de simulations automatisées, scriptées ou manuelles.

1.2 Objectifs

- Créer un outil de test fiable et précis pour les Game Designers.
- Concevoir une architecture hautement modulaire permettant d'implémenter, d'activer ou de désactiver des règles rapidement.
- Générer des comptes rendus exhaustifs pour chaque "Run", capturant l'état des données à un instant précis.
- Analyser les possibilités et les "edge cases" (cas limites) de l'équilibrage grâce à l'Intelligence Artificielle.

1.3 Cibles utilisateurs

- **Game Designers** : Utilisent l'outil pour tester l'équilibrage pour la création de monstres, modifier des règles de jeu, ou encore d'équilibrer des statistiques
- **Développeurs** : Intègrent de nouveaux modules de règles et configurent les scripts de test.

2 Architecture Technique

2.1 Stack technologique

Couche	Technologie	Version
Interface Graphique	Qt	6.8
Core app	CPP	C++23
Script IA	Python	3.15
Stockage de données	JSON	-

2.2 Architecture applicative

```
Danjon Simulator/
|
|---- front/          # Framework Qt
|   |
|   |---- front/src/    # Code source UI
|   |---- front/include/ # Headers de l'UI
|   |---- front/assets/  # Icônes, feuilles de style QSS, fichiers de ressources...
|
|---- back/           # Logique des systèmes C++
|   |
|   |---- back/core/    # Moteur principal (Boucle de simulation, gestionnaire d'Entités)
|   |---- back/systems/ # Les modules de règles
|   |---- back/bindings/ # Communication entre le python et C++ Ou entre une source externe et C++ pour l'IA
|
|---- tests/          # tests automatisé vérifiant que les systèmes fonctionne bien
|
|---- data/           # Fichiers JSON contenant chacune des données
|   |
|   |---- entities/     # Archétypes, statistiques de base, races
|   |---- systems/      # Règles du jeu (Poison, Saignement, Endurance...)
|
|---- logs/           # Les comptes rendu des anciens run
|
|---- CMakeLists.txt  # Fichier racine pour compiler le projet
```

2.3 Communication

Front -> Back : Appels aux méthodes C++ lors d'inputs par l'utilisateur.

Back -> Data : Lecture des fichiers locaux JSON, permettant d'initialiser les systèmes aux bonnes valeurs

Tests -> Back : Test des objets systèmes, de lancement de simulation.

Front -> Logs : Compte rendu des anciens runs dans le logiciel

Tests -> Logs : Regarder si le run s'est bien exécuté

3 Fonctionnalités Utilisateur

3.1 Page de Run

3.1.1 Choisir entre les différents profils entre les deux camps

Une liste déroulante pouvant choisir entre :

- **Entrée manuelle** : A chaque "Tour" on a un choix entre **Attaquer**, **Parer**, **Esquiver**, **Magie**, **Passer**. Pour terminer un tour, il faut que le nombres d'action défini par le système soit égale à 0 et à partir de la choisir entre "Surcadencer" qui est un système permettant à des couts plus élevés à jouer une fois de plus, soit de "Finir le

tour”. Lorsque les actions des 2 camps ont été défini, le tour se calcul avant de passer au suivant.

- **Script** : A chaque tour, on reçoit une liste des informations sur son personnage, ainsi que celui de l’adversaire, avec ces informations, le script doit return le code correspondant à une des actions manuelles défini ci-dessus.
- **Externe** : En liaison TCP, fonctionne de la même manière qu’un script mais de manière externe avec un temps de réponse devant être < 10 secondes. Parfait pour l’implémentation de modèle d’IA. Afin d’éviter des problèmes d’erreurs, par défaut la réponse est chargée en cas de timeout de “Passer” ou “Finir le tour” selon la situation

3.1.3 Choix d’un personnage

Permet de choisir à partir d’une liste déroulante une liste de personnage. Il peut être de type monstre ou type racial. Chacun ayant leur particularité.

3.1.4 Lancer un run

Un bouton permettant de lancer un run. Nécessite de définir 1 personnage de chaque côté.

3.2 Page d’historique

Cette page est nécessaire afin de pouvoir consulter les runs effectués, et permettre de les analyser et comparer.

4 Gestions d’erreurs

4.1 JSON corrompu

Si des changements dans un **JSON** mène à un souci technique, un bandeau rouge doit apparaître sur l’application affichant quel fichier est en défaut, obtenant un maximum de détails empêchant les runs d’être possible tant qu’il n’est pas résolu.

4.2 Crash lors d’un Run

Si un run en cours plante, le **run** doit se terminer remplissant le fichier de **log** par la cause du plantage, avec si possible des informations sur l’endroit qui coince.

5 Systèmes à implémenter

5.1 Personnages

5.1.1 Statistiques

Chaque Personnages, possèdent des statistiques qui lui sont propre selon 5 caractéristiques, ces statistiques augmentent avec leur niveau et leur rang. Les personnages possèdent des stats de base, les stats actuelles sont calculées par la formule

Force / Résistance / Vitesse / Force Magique / Résistance Magique

5.1.2 Passifs

Certains personnages peuvent avoir des passifs, qui sont actif sous certaines conditions ou non

5.1.3 Réserve physique

Les personnages possèdent une réserve physique, défini par rapport à son rang.

5.1.4 Magie

Certains personnages sont magiques, et donc possède une magie que ce soit de soin, utilitaire, ou offensive. Cette magie possède une réserve calculée par rapport au rang du personnage.

5.2 Différences statistiques

Ce système permet de convertir l'écart mathématique brut entre deux statistiques en opposition (ex: Attaque de l'entité A contre Défense de l'entité B) en un **degré d'efficacité** qualitatif. Ce degré est mis à l'échelle dynamiquement en fonction du niveau des personnages. L'échelle mesure une efficacité de nul à domination (-7 à +7).

5.3 Dommages

Les dommages sont basé par l'échelle -7 +7 de la différence statistique. On prend en compte que la mort d'un personnage est à +5. Pour augmenter d'un stade, il faut avoir reçu 2 fois le stade précédent. À +4 est la seule exception où un +0 suffit pour en finir.

À partir de +3, les statistiques hors résistance et résistance magique baissent de 15%

A Partir de +4, les statistiques hors résistance et résistance magique baissent de 30%

5.4 Armes, Armures et Objets

5.4.1 Dégâts des armes

Les dégâts des armes dépendent de leur type. L'augmentation est représentée par un +1 sur la ligne des différences statistiques. Chaque coup réduit la durabilité de l'objet.

Tranchant : Les armes tranchantes augmentent les dégâts de l'utilisateur par rapport au tranchant de celui-ci. L'augmentation est représentée par un -1 sur la ligne des différences statistiques.

Contondant : Les armes contondantes augmentent les dégâts de l'utilisateur par rapport à la dureté de celui-ci.

5.4.2 Résistance des armures

L'armure permet de résister aux dégâts, chaque type d'armure a des caractéristiques différentes. Chaque coup reçu réduit la durabilité de l'objet. La valeur de la dureté de l'équipement permet de réduire du nombre de stats correspondant le nombre de ligne de différences statistiques. Si l'armure est détruite par le coup, les dommages sont non réduits.

- **Armure en cuir/écaille** : Réduction des dégâts reçus de 15%.
- **Armure en métal, chitine ou cristal** : Réduction totale des dommages tranchants, réduction des dégâts contondants reçus de 15%.

5.4.3 Durabilité et Usure

La durabilité d'un objet dépend de l'objet créé ainsi que le niveau de son matériau, ainsi que la qualification de la personne qui crée l'objet. Pour des facilités de la première version, la durabilité sera donnée en brut et non calculé. Selon le composant de l'objet, certains effets peuvent augmenter l'usure

- **Tranchant physique** : Double l'usure sur les tissus, le cuir et les écailles.
- **Contondant** : Double l'usure sur la chitine, le métal, la glace et le cristal.
- **Ignition** : Inflige -20 sur le tissu, le cuir ou les écailles.
- **Corrosion** : Inflige -20 sur le métal ou la chitine.

5.4.4 Catalyseur

Les catalyseurs sont une sphère de la taille d'une bille pour les plus faibles, pouvant contenir une magie. Sa réserve et sa puissance dépend du rang du catalyseur. Ils peuvent être introduits dans un autre objet.

5.5 Magies

Les magies sont soit contenu dans un catalyseur, soit dans un personnage. Pour des choix de simulation “simplifié”, un personnage ne peut avoir qu’une magie à la fois, et seuls ceux des catégories ci-dessous facilement “chiffable” y apparaissent d’une puissance moyenne. Certaines magies peuvent être de plusieurs catégories. Chaque magie à un coût défini.

5.5.1 Magie Offensive

Inflige des dégâts magique correspondant à la force magique de l’utilisateur ou du catalyseur multiplié par un ratio.

5.5.2 Magie Boost

Augmente une/des statistiques d’une valeur dépendant de la force magique de l’utilisateur ou du catalyseur multiplié par un ratio.

5.5.3 Magie Soins

Soigne les dommages subis. Plus le soin est puissant, plus il permet de soigner des blessures graves.

- Très faible : Blessures de -7 à -2
- Faible : Blessures de -7 à 0
- Moyen : Blessures de -7 à +2
- Fort : Blessures de -7 à 3
- Très Fort : Blessures de -7 à 4
- Extrême : Régénère tout, y compris le sang

5.6 Saignements

Le saignement est un système permettant d’empêcher les combats à rallonge. Chaque personnage à une valeur de 32 points de sang et en perd selon la gravité de la blessure. Seuls les dégâts tranchant le permettent, magique comme physique.

- **Faible** : -1 / tours
- **Moyen** : -2 / tours
- **Fort** : -5 / tours

5.7 Parade

Réduit les dégâts de 10%, augmenté à 100% si équipé d’un bouclier en métal pour bloquer une attaque tranchante. Certains passifs peuvent monter les dégâts réduits.

5.8 Poids

Un sélecteur de poids entre Léger / Moyen / Lourd / Surpoids / Effondrement, modifiant la vitesse du personnage.

5.9 Classes

Un sélecteur non obligatoire pour certains personnages pour avoir une classe attribuée, modifiant légèrement certaines règles, comme un tour d'esquive supplémentaire, subir les malus que plus tard...

6 Gestion des versions du logiciel / Source control

Le projet sera hébergé sur un dépôt public **GitHub**, garantissant une totale transparence et facilitant le suivi pour l'équipe de Game Design. Le déploiement des versions stables s'appuiera sur un système de **Tags** rigoureux.

La nomenclature des mises à jour suivra la logique suivante :

- **Versions Majeures (ex : v1.0.0, v2.0.0)** : Calquées sur les évolutions majeures du serveur "Danjon". Elles interviendront lors de changements profonds dans les règles RP ou lors d'une refonte structurelle des systèmes du simulateur.
- **Versions Intermédiaires (ex : v1.1.0, v1.2.0)** : Déployées pour intégrer de nouvelles fonctionnalités (features) à l'outil ou pour déployer des correctifs urgents sur des bugs bloquants.
- **Versions Mineures (ex : v1.1.1, v1.1.2)** : Réservées aux correctifs de bugs non bloquants (hotfixes), aux petites optimisations de performance et aux ajustements mineurs d'interface.